NATURAL LANGUAGE PROCESSING LABORATORY MANUAL

Classical NLP Using Python

Text Input, Processing, Linguistic Analysis and Machine Learning

Scope: Pre-LLM NLP Methods

By Dr. Ujwala Bharambe

Table of Contents

Sections 1–10 (Foundations)**
1. Course Overview
2. Course Objectives
3. Learning Outcomes
4. Evolution of NLP (4.1 Rule-Based, 4.2 Statistical, 4.3 ML-Based, 4.4 Neural Pre-LLM)
5. General NLP Processing Pipeline (Stages 1–10)
6. Python Programming Approach for NLP
7. Python Data Structures (7.1 String – 7.8 Sparse Matrix)
8. Important Python Libraries (8.1 Built-in – 8.9 Gensim)
9. Laboratory Requirements
10. General Experiment Format

Experiments 1–16
1. Working with Text Input and Python Data Structures
2. Text Cleaning Using Python and Regular Expressions
3. Sentence and Word Tokenisation
4. Stop-Word Removal and Word-Frequency Analysis
5. Stemming and Lemmatisation
6. Part-of-Speech Tagging
7. Chunking and Phrase Extraction
8. Named-Entity Recognition
9. Dependency Parsing
10. Bag-of-Words Representation
11. N-Gram Language Features
12. TF-IDF Text Representation
13. Text Similarity Using Cosine Similarity
14. Text Classification Using Naive Bayes
15. Comparison of Classical Text Classifiers
16. End-to-End Text-Processing Pipeline

Sections 11–23 (Projects, Reference & Assessment)

11. Suggested Mini Projects (Medical reports, News classification, Feedback analysis, Spam detection, Resume extraction, Notice extraction, FAQ retrieval)
12. General Dataset Structure
13. Dataset Inspection Program
14. Common NLP Errors (14.1–14.6)
15. Model Evaluation (Confusion Matrix, Accuracy, Precision, Recall, F1)
16. Error Analysis
17. Suggested Laboratory Sequence (15-week plan)
18. Laboratory Assessment Scheme
19. Journal-Writing Format
20. General Viva Questions (30 questions)
21. Final Integrated NLP Workflow

Course Overview

Natural Language Processing enables computers to collect, clean, organize, analyse and interpret human language.

This laboratory focuses on the classical NLP workflow:

Text Input → Cleaning → Tokenisation → Linguistic Processing → Feature Extraction → Machine Learning → Evaluation

The laboratory does not use large language models, prompt engineering, retrieval-augmented generation or generative AI.

2. Course Objectives

After completing the laboratory, students should be able to:

1. Read text from different input sources.
2. Represent text using appropriate Python data structures.
3. Clean and normalise unstructured textual data.
4. Perform sentence and word tokenisation.
5. Remove stop words and punctuation.
6. Apply stemming and lemmatisation.
7. Perform part-of-speech tagging.
8. Identify named entities.
9. Perform syntactic and dependency analysis.
10. Generate Bag-of-Words, n-gram and TF-IDF representations.
11. Calculate text similarity.
12. Build classical text-classification systems.
13. Evaluate NLP models using suitable metrics.
14. Design an end-to-end classical NLP application.

3. Learning Outcomes

Students will be able to:

- Identify the stages of an NLP pipeline.
- Select suitable NLP libraries.
- Work with structured, semi-structured and unstructured text.
- Connect linguistic concepts with Python implementation.
- Use classical machine-learning techniques for text analysis.
- Interpret the output of NLP models.
- Compare alternative preprocessing and feature-engineering methods.
- Develop reproducible NLP experiments.

4. Evolution of NLP

4.1 Rule-Based NLP

Early NLP systems used manually written language rules. Examples include:

- Dictionary lookup
- Grammar rules
- Regular expressions

- Hand-crafted patterns
- Finite-state machines

Example: Identifying an Email Address

```
import re

text = "Contact us at support@example.com"

emails = re.findall(
    r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}",
    text
)

print(emails)
```

Strengths

- Easy to understand
- Useful for well-defined patterns
- Does not require large datasets

Limitations

- Difficult to scale
- Rules become complex
- Cannot easily handle language ambiguity

4.2 Statistical NLP

Statistical NLP learns language patterns from text collections. Important methods include:

- Word frequency
- Probability models
- N-gram language models
- Hidden Markov Models
- Naive Bayes
- Conditional Random Fields

4.3 Machine-Learning-Based NLP

Text is converted into numerical features and given to a machine-learning algorithm.

Common Models

- Naive Bayes
- Logistic Regression
- Decision Tree
- Random Forest
- Support Vector Machine
- K-Nearest Neighbours

Common Text Representations

- Bag-of-Words
- Binary word occurrence
- Word frequency
- N-grams
- TF-IDF

4.4 Neural NLP Before Large Language Models

Before modern LLMs, neural NLP commonly used:

- Word2Vec
- GloVe
- FastText
- Recurrent neural networks
- LSTM
- GRU
- Convolutional neural networks for text
- Sequence-to-sequence models
- Attention mechanisms
- Early transformer models such as BERT

This laboratory mainly focuses on rule-based, statistical and classical machine-learning NLP.

5. General NLP Processing Pipeline

Stage 1: Input Collection

Text may be obtained from:

- Keyboard input
- Text files
- CSV files
- JSON files
- XML files
- Databases
- Web pages
- Social-media datasets
- Documents
- Application logs

Stage 2: Data Inspection

Check:

- Number of documents
- Missing values
- Duplicate records

- Encoding
- Language
- Document length
- Class distribution

Stage 3: Text Cleaning

Typical operations:

- Convert text to lowercase
- Remove HTML tags
- Remove URLs
- Remove email addresses
- Remove punctuation
- Remove numbers
- Remove unwanted spaces
- Expand contractions
- Handle emojis
- Correct spelling where necessary

Stage 4: Tokenisation

Split text into:

- Sentences
- Words
- Subwords, where required

Stage 5: Normalisation

Apply:

- Lowercase conversion
- Stemming
- Lemmatisation
- Spelling normalisation
- Contraction expansion

Stage 6: Linguistic Analysis

Apply:

- Part-of-speech tagging
- Chunking
- Named-entity recognition
- Parsing
- Dependency analysis

Stage 7: Feature Extraction

Convert text into numerical form using:

- Word counts
- Bag-of-Words
- N-grams
- TF-IDF
- Word embeddings

Stage 8: Modelling

Train a classifier, clustering algorithm or sequence model.

Stage 9: Evaluation

Evaluate using:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix
- Cross-validation

Stage 10: Interpretation

Study:

- Important features
- Prediction errors
- Class imbalance
- Linguistic patterns
- Model limitations

6. Python Programming Approach for NLP

Python is widely used in NLP because it provides:

- Simple string handling
- Flexible data structures
- Regular-expression support
- Data-analysis libraries
- Machine-learning libraries
- Specialised NLP packages

7. Python Data Structures Used in NLP

7.1 String

A string represents a single text sequence.

```
sentence = "Natural Language Processing is interesting."
print(type(sentence))
```

Common string operations:

```
text = "Natural Language Processing"

print(text.lower())
print(text.upper())
print(text.split())
print(text.replace("Language", "Text"))
print(len(text))
```

7.2 List

A list is used to store ordered tokens, sentences or documents.

```
tokens = ["natural", "language", "processing"]
print(tokens[0])
print(len(tokens))
```

A list can contain duplicate items.

```
words = ["data", "text", "data", "language"]
```

7.3 Tuple

A tuple stores fixed ordered information.

```
tagged_word = ("processing", "NOUN")
print(tagged_word[0])
print(tagged_word[1])
```

Tuples are frequently used for:

- Word-tag pairs
- Token-position pairs
- Entity-label pairs

7.4 Dictionary

A dictionary stores information using key-value pairs.

```
word_frequency = {
    "data": 5,
    "language": 3,
    "model": 2
}

print(word_frequency["data"])
```

Dictionaries are useful for:

- Word frequency
- Vocabulary mapping
- Class labels

- Document metadata
- Feature storage

7.5 Set

A set stores unique elements.

```
words = ["data", "text", "data", "language"]
unique_words = set(words)

print(unique_words)
```

Sets are useful for:

- Vocabulary generation
- Stop-word removal
- Comparison of unique words

7.6 NumPy Array

NumPy arrays store numerical text representations.

```
import numpy as np

vector = np.array([1, 0, 3, 2])
print(vector)
```

7.7 Pandas DataFrame

A DataFrame stores a dataset in rows and columns.

```
import pandas as pd

data = {
    "text": [
        "The service was good",
        "The product was poor"
    ],
    "label": ["positive", "negative"]
}

df = pd.DataFrame(data)
print(df)
```

7.8 Sparse Matrix

Text datasets contain large vocabularies, but each document uses only a small number of words. Therefore, Bag-of-Words and TF-IDF are usually represented using sparse matrices.

```
from scipy.sparse import csr_matrix

matrix = csr_matrix([
    [1, 0, 2],
    [0, 3, 0]
```

```
])

print(matrix)
```

8. Important Python Libraries

8.1 Built-in Python

Used for:

- Strings
- Lists
- Dictionaries
- Files
- Functions
- Exception handling

8.2 re

Used for regular expressions and pattern matching. Applications:

- URL removal
- Email detection
- Number extraction
- Punctuation cleaning
- Pattern-based information extraction

```
import re
```

8.3 NLTK

Natural Language Toolkit supports:

- Tokenisation
- Stemming
- Lemmatisation
- Stop words
- POS tagging
- Chunking
- Corpora
- WordNet

```
import nltk
```

8.4 spaCy

spaCy supports:

- Industrial-strength tokenisation

- POS tagging
- Named-entity recognition
- Dependency parsing
- Sentence segmentation
- Linguistic features

```
import spacy
```

8.5 Pandas

Used for:

- Reading datasets
- Handling missing values
- Filtering data
- Creating new text columns
- Grouping and summarising records

8.6 NumPy

Used for:

- Numerical arrays
- Vector operations
- Similarity calculations
- Mathematical transformations

8.7 Scikit-learn

Used for feature extraction, classification, clustering, model evaluation, train-test splitting and pipelines. Important classes:

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
```

8.8 BeautifulSoup

Used for extracting and cleaning web-page text.

```
from bs4 import BeautifulSoup
```

8.9 Gensim

Traditionally used for:

- Word2Vec
- Topic modelling
- Document similarity

- Corpus processing

9. Laboratory Requirements

Software

- Python 3.x
- Jupyter Notebook or Google Colab
- NLTK
- spaCy
- Pandas
- NumPy
- Scikit-learn
- Matplotlib
- BeautifulSoup
- Gensim, where required

Installation

```
pip install nltk spacy pandas numpy scikit-learn matplotlib beautifulsoup4 gensim
```

Install an English spaCy model:

```
python -m spacy download en_core_web_sm
```

Basic NLTK downloads:

```
import nltk

nltk.download("punkt")
nltk.download("stopwords")
nltk.download("wordnet")
nltk.download("averaged_perceptron_tagger")
nltk.download("maxent_ne_chunker")
nltk.download("words")
```

10. General Experiment Format

Each laboratory experiment should contain:

15. Experiment number
16. Title
17. Aim
18. Learning objectives
19. Theory
20. Input
21. Algorithm
22. Python program
23. Output

24. Observations
25. Result
26. Questions
27. Extension activity

EXPERIMENT 1

Working with Text Input and Python Data Structures

Aim

To read text from different input sources and represent it using Python data structures.

Learning Objectives

Students will learn to:

- Accept keyboard input
- Read a text file
- Read CSV and JSON data
- Store text as strings, lists, tuples and dictionaries
- Inspect a textual dataset

Theory

Text may exist as one sentence, one document, multiple documents, document-label pairs, structured records, or nested JSON objects.

The selected data structure depends on the NLP task.

Program 1: Keyboard Input

```
text = input("Enter a sentence: ")

print("Original text:", text)
print("Number of characters:", len(text))
print("Number of words:", len(text.split()))
```

Program 2: Read a Text File

```
file_path = "sample.txt"

try:
    with open(file_path, "r", encoding="utf-8") as file:
        text = file.read()

    print(text)

except FileNotFoundError:
    print("The specified file was not found.")

except UnicodeDecodeError:
    print("Unable to decode the file.")
```

Program 3: Read Line by Line

```
documents = []
```

```
with open("sample.txt", "r", encoding="utf-8") as file:
    for line in file:
        cleaned_line = line.strip()

        if cleaned_line:
            documents.append(cleaned_line)

print(documents)
```

Program 4: Read CSV Data

```
import pandas as pd

df = pd.read_csv("reviews.csv")

print(df.head())
print(df.columns)
print(df.shape)
print(df.isnull().sum())
```

Program 5: Read JSON Data

```
import json

with open("documents.json", "r", encoding="utf-8") as file:
    data = json.load(file)

print(data)
```

Suggested Input

Natural Language Processing helps computers analyse human language. Python provides useful tools for processing text.

Result

Text was successfully collected from different input sources and represented using appropriate Python data structures.

Questions

28. What is the difference between a string and a list?
29. Why is UTF-8 encoding important?
30. When should a dictionary be used in NLP?
31. Why are DataFrames suitable for text-classification datasets?
32. What is the purpose of exception handling?

Extension Activity

- Create a DataFrame containing: document ID, document text, category, number of words, and number of characters.

EXPERIMENT 2

Text Cleaning Using Python and Regular Expressions

Aim

To clean and normalise raw textual data.

Learning Objectives

Students will learn to:

- Remove URLs
- Remove HTML tags
- Remove punctuation
- Remove digits
- Remove email addresses
- Remove extra spaces
- Remove unwanted symbols

Theory

Raw text may contain noise such as: " Visit https://example.com NOW!!! Contact: info@example.com "

Cleaning improves consistency but must be performed according to the application. For example, numbers may be important in financial text, punctuation may be important in sentiment analysis, and hashtags may be important in social-media analysis.

Program

```
import re
from bs4 import BeautifulSoup

def clean_text(text):
    if not isinstance(text, str):
        return ""

    # Remove HTML
    text = BeautifulSoup(text, "html.parser").get_text(" ")

    # Convert to lowercase
    text = text.lower()

    # Remove URLs
    text = re.sub(r"http\S+|www\.\S+", " ", text)

    # Remove email addresses
    text = re.sub(
        r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}",
        " ",
```

```
        text
    )

    # Remove digits
    text = re.sub(r"\d+", " ", text)

    # Remove punctuation and special symbols
    text = re.sub(r"[^a-z\s]", " ", text)

    # Remove extra spaces
    text = re.sub(r"\s+", " ", text).strip()

    return text


sample = """
<p>Visit https://example.com NOW!!!
Contact info@example.com. Offer valid till 2026.</p>
"""

print("Original:")
print(sample)

print("\nCleaned:")
print(clean_text(sample))
```

Output

```
visit now contact offer valid till
```

Observation

Cleaning reduced noise and produced normalised text. However, removal rules may also remove useful information.

Result

Raw text was cleaned using Python string operations, regular expressions and HTML parsing.

Questions

33. Why should URLs be removed?
34. When should numbers be retained?
35. What is the difference between "re.sub()" and "str.replace()"?
36. Why can aggressive cleaning be harmful?
37. Should all text always be converted to lowercase?

Extension Activity

- Modify the program to preserve currency values, percentages, hashtags, dates and negation words.

EXPERIMENT 3

Sentence and Word Tokenisation

Aim

To divide text into sentences and words.

Theory

Tokenisation converts continuous text into smaller units called tokens.

Sentence tokenisation

"NLP is useful. It helps analyse language." becomes: ["NLP is useful.", "It helps analyse language."]

Word tokenisation

"NLP is useful." becomes: ["NLP", "is", "useful", "."]

Program Using NLTK

```
import nltk
from nltk.tokenize import sent_tokenize, word_tokenize

text = """
Natural Language Processing is a branch of artificial intelligence.
It helps computers understand human language.
"""

sentences = sent_tokenize(text)
words = word_tokenize(text)

print("Sentences:")
for sentence in sentences:
    print(sentence)

print("\nWords:")
print(words)
```

Program Using spaCy

```
import spacy

nlp = spacy.load("en_core_web_sm")

text = """
Natural Language Processing is useful.
It supports many real-world applications.
"""

doc = nlp(text)

print("Sentences:")
for sentence in doc.sents:
```

```
    print(sentence.text)

print("\nTokens:")
for token in doc:
    print(token.text)
```

Comparison Activity

Compare the output of Python split(), NLTK word_tokenize(), and spaCy tokenisation:

```
text = "Dr. Rao can't attend today's NLP class."

print(text.split())
```

Result

Text was successfully separated into sentence and word tokens.

Questions

38. Why is "split()" not sufficient for proper tokenisation?
39. How are contractions handled?
40. Is punctuation considered a token?
41. What problems occur with abbreviations?
42. Why is tokenisation language-dependent?

EXPERIMENT 4

Stop-Word Removal and Word-Frequency Analysis

Aim

To remove common words and calculate word frequencies.

Theory

Stop words are high-frequency functional words such as the, is, are, of, to, in.

Removing stop words can reduce feature size. However, stop-word removal is not always suitable — for sentiment analysis, words such as "not" may be important.

Program

```
from collections import Counter
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

text = """
Natural language processing is useful because it helps computers
process and analyse natural language.
```

```
"""

tokens = word_tokenize(text.lower())

stop_words = set(stopwords.words("english"))

filtered_tokens = [
    token
    for token in tokens
    if token.isalpha() and token not in stop_words
]

frequency = Counter(filtered_tokens)

print("Original tokens:")
print(tokens)

print("\nFiltered tokens:")
print(filtered_tokens)

print("\nWord frequencies:")
for word, count in frequency.most_common():
    print(word, count)
```

Alternative Using Dictionary

```
frequency = {}

for word in filtered_tokens:
    frequency[word] = frequency.get(word, 0) + 1

print(frequency)
```

Result

Stop words were removed and the frequency of meaningful words was calculated.

Questions

43. What is a stop word?
44. Why should negation words sometimes be retained?
45. What is the difference between a dictionary and "Counter"?
46. What is vocabulary size?
47. How can domain-specific stop words be created?

Extension Activity

- Create a customised stop-word list for academic documents, medical reports, product reviews and news articles.

EXPERIMENT 5

Stemming and Lemmatisation

Aim

To reduce different word forms to a common base form.

Theory

Stemming removes word endings using rules: connected → connect, connecting → connect, studies → studi. A stem may not be a valid dictionary word.

Lemmatisation identifies the dictionary form of a word: studies → study, better → good, running → run. Lemmatisation usually requires grammatical information.

Program

```
from nltk.stem import PorterStemmer, LancasterStemmer
from nltk.stem import WordNetLemmatizer

words = [
    "connect", "connected", "connecting", "connection",
    "studies", "studying", "better"
]

porter = PorterStemmer()
lancaster = LancasterStemmer()
lemmatizer = WordNetLemmatizer()

print(
    f"{'Word':<15}"
    f"{'Porter':<15}"
    f"{'Lancaster':<15}"
    f"{'Lemma':<15}"
)

for word in words:
    print(
        f"{word:<15}"
        f"{porter.stem(word):<15}"
        f"{lancaster.stem(word):<15}"
        f"{lemmatizer.lemmatize(word):<15}"
    )
```

POS-Aware Lemmatisation

```
from nltk.corpus import wordnet
from nltk import pos_tag
from nltk.tokenize import word_tokenize

def get_wordnet_pos(tag):
    if tag.startswith("J"):
        return wordnet.ADJ
    if tag.startswith("V"):
        return wordnet.VERB
    if tag.startswith("N"):
        return wordnet.NOUN
```

```
    if tag.startswith("R"):
        return wordnet.ADV

    return wordnet.NOUN


sentence = "The students were studying better methods."

tokens = word_tokenize(sentence)
tagged_tokens = pos_tag(tokens)

lemmas = []

for word, tag in tagged_tokens:
    lemma = lemmatizer.lemmatize(
        word.lower(),
        get_wordnet_pos(tag)
    )

    lemmas.append(lemma)

print(lemmas)
```

Result

Words were normalised using stemming and lemmatisation.

Questions

48. What is the difference between a stem and a lemma?
49. Why can stemming produce invalid words?
50. Which is computationally faster?
51. Why does lemmatisation require POS information?
52. Which method is more suitable for search applications?

EXPERIMENT 6

Part-of-Speech Tagging

Aim

To assign grammatical categories to words.

Theory

Part-of-speech tags identify the grammatical role of each word, such as noun, verb, adjective, adverb, pronoun, preposition, or conjunction.

Example: "Students learn NLP." → Students/NOUN learn/VERB NLP/NOUN

Program Using NLTK

```
import nltk
from nltk.tokenize import word_tokenize
from nltk import pos_tag

sentence = "The intelligent student solved the difficult problem quickly."

tokens = word_tokenize(sentence)
tagged_words = pos_tag(tokens)

for word, tag in tagged_words:
    print(f"{word:<15} {tag}")
```

Program Using spaCy

```
import spacy

nlp = spacy.load("en_core_web_sm")

sentence = "The intelligent student solved the difficult problem quickly."

doc = nlp(sentence)

for token in doc:
    print(
        f"{token.text:<15}"
        f"{token.pos_:<12}"
        f"{token.tag_:<10}"
    )
```

Result

Each token was assigned a grammatical category.

Questions

53. What is the difference between "POS" and a detailed tag?
54. Can the same word have different tags?
55. How does context affect tagging?
56. What is the tag of "book" in different sentences?
57. How is POS tagging useful in information extraction?

Extension Activity

Compare: "I will book a ticket." vs. "This is an interesting book."

EXPERIMENT 7

Chunking and Phrase Extraction

Aim

To identify noun phrases and other meaningful word groups.

Theory

Chunking performs shallow parsing.

Example: "The intelligent student solved a difficult problem." Possible noun phrases: "the intelligent student", "a difficult problem".

Program Using NLTK

```
import nltk
from nltk import pos_tag
from nltk.tokenize import word_tokenize

sentence = "The intelligent student solved a difficult NLP problem."

tokens = word_tokenize(sentence)
tagged_tokens = pos_tag(tokens)

grammar = r"""
    NP: {<DT>?<JJ.*>*<NN.*>+}
"""

chunk_parser = nltk.RegexpParser(grammar)
tree = chunk_parser.parse(tagged_tokens)

print(tree)

for subtree in tree.subtrees():
    if subtree.label() == "NP":
        phrase = " ".join(word for word, tag in subtree.leaves())
        print("Noun phrase:", phrase)
```

Program Using spaCy Noun Chunks

```
import spacy

nlp = spacy.load("en_core_web_sm")

doc = nlp(
    "The intelligent student solved a difficult NLP problem."
)

for chunk in doc.noun_chunks:
    print(chunk.text)
```

Result

Noun phrases were identified using POS patterns and chunking.

Questions

58. What is shallow parsing?
59. How does chunking differ from full parsing?

60. What is a noun phrase?
61. What does "<JJ.*>*" represent?
62. How can chunking support information extraction?

EXPERIMENT 8

Named-Entity Recognition

Aim

To identify names, places, organisations, dates and other entities.

Theory

Named-entity recognition identifies real-world entities.

Common Entity Classes

- PERSON
- ORGANIZATION
- LOCATION
- DATE
- TIME
- MONEY
- PERCENT
- PRODUCT

Program Using spaCy

```
import spacy

nlp = spacy.load("en_core_web_sm")

text = """
Sundar Pichai spoke at Google headquarters in California
on Monday. The company announced an investment of $10 million.
"""

doc = nlp(text)

for entity in doc.ents:
    print(
        f"{entity.text:<25}"
        f"{entity.label_:<12}"
        f"{spacy.explain(entity.label_)}"
    )
```

Store Entities in a Dictionary

```
entities = {}

for entity in doc.ents:
    entities.setdefault(entity.label_, []).append(entity.text)

print(entities)
```

Result

Named entities were extracted and grouped by their entity labels.

Questions

63. What is a named entity?
64. What is the difference between a noun and a named entity?
65. Why can domain-specific entities be difficult to identify?
66. What entities occur in medical reports?
67. How can NER support document indexing?

Extension Activity

- Extract entities from a financial-news article, a college notice, a medical report and a sports article.

EXPERIMENT 9

Dependency Parsing

Aim

To identify grammatical relationships between words.

Theory

Dependency parsing represents relationships such as subject, object, modifier, root verb, and auxiliary verb.

Consider: "The student completed the assignment." Important relations: completed = root; student = subject of completed; assignment = object of completed.

Program

```
import spacy

nlp = spacy.load("en_core_web_sm")

sentence = "The student completed the NLP assignment successfully."

doc = nlp(sentence)

print(
    f"{'Token':<15}"
    f"{'Dependency':<15}"
```

```
    f"{'Head':<15}"
    f"{'POS':<10}"
)

for token in doc:
    print(
        f"{token.text:<15}"
        f"{token.dep_:<15}"
        f"{token.head.text:<15}"
        f"{token.pos_:<10}"
    )
```

Extract Subject-Verb-Object

```
subject = None
verb = None
obj = None

for token in doc:
    if token.dep_ in ("nsubj", "nsubjpass"):
        subject = token.text

    if token.dep_ == "ROOT":
        verb = token.text

    if token.dep_ in ("dobj", "obj"):
        obj = token.text

print("Subject:", subject)
print("Verb:", verb)
print("Object:", obj)
```

Result

Dependency relations between words were identified.

Questions

68. What is the root of a sentence?
69. What is the role of the subject?
70. How does dependency parsing differ from POS tagging?
71. Why is parsing difficult for complex sentences?
72. How can dependency parsing support relation extraction?

EXPERIMENT 10

Bag-of-Words Representation

Aim

To convert text documents into word-count vectors.

Theory

Bag-of-Words represents each document using the occurrence or frequency of vocabulary words. The model ignores word order.

Example — D1: "NLP processes language", D2: "NLP processes text". Vocabulary: language, NLP, processes, text.

Program

```
from sklearn.feature_extraction.text import CountVectorizer
import pandas as pd

documents = [
    "NLP processes language",
    "NLP processes text",
    "Language processing uses text"
]

vectorizer = CountVectorizer()

bow_matrix = vectorizer.fit_transform(documents)

feature_names = vectorizer.get_feature_names_out()

bow_df = pd.DataFrame(
    bow_matrix.toarray(),
    columns=feature_names
)

print(bow_df)
```

Binary Bag-of-Words

```
binary_vectorizer = CountVectorizer(binary=True)

binary_matrix = binary_vectorizer.fit_transform(documents)

binary_df = pd.DataFrame(
    binary_matrix.toarray(),
    columns=binary_vectorizer.get_feature_names_out()
)

print(binary_df)
```

Important Parameters

```
CountVectorizer(
    lowercase=True,
    stop_words="english",
    min_df=1,
    max_df=1.0,
    binary=False
)
```

Result

Documents were converted into numerical word-count vectors.

Questions

73. What is a vocabulary?
74. Why does Bag-of-Words ignore word order?
75. What is a document-term matrix?
76. Why is the matrix sparse?
77. What is the difference between count and binary representation?

EXPERIMENT 11

N-Gram Language Features

Aim

To represent text using sequences of words.

Theory

An n-gram is a sequence of "n" consecutive tokens.

Unigram: natural, language, processing. Bigram: natural language, language processing. Trigram: natural language processing.

N-grams capture limited word order.

Program

```
from sklearn.feature_extraction.text import CountVectorizer
import pandas as pd

documents = [
    "machine learning is useful",
    "deep learning is useful",
    "machine learning processes data"
]

vectorizer = CountVectorizer(
    ngram_range=(1, 2)
)

matrix = vectorizer.fit_transform(documents)

df = pd.DataFrame(
    matrix.toarray(),
    columns=vectorizer.get_feature_names_out()
)

print(df)
```

Manual Bigram Generation

```
from nltk.util import ngrams
from nltk.tokenize import word_tokenize

sentence = "natural language processing is useful"

tokens = word_tokenize(sentence)

bigrams = list(ngrams(tokens, 2))
trigrams = list(ngrams(tokens, 3))

print("Bigrams:", bigrams)
print("Trigrams:", trigrams)
```

Result

Unigrams, bigrams and trigrams were generated and represented numerically.

Questions

78. What information do bigrams retain?
79. What is the disadvantage of large n-grams?
80. Why does vocabulary size increase?
81. How can bigrams help sentiment analysis?
82. Why is "not good" more informative than separate words?

EXPERIMENT 12

TF-IDF Text Representation

Aim

To calculate the importance of words across documents.

Theory

Term Frequency–Inverse Document Frequency gives higher weight to words that occur frequently in a document but occur in fewer documents across the collection.

TF(t,d) = (Frequency of term t in document d) / (Total terms in document d)

IDF(t) = log(N / DF(t))

TFIDF(t,d) = TF(t,d) × IDF(t)

Program

```
from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd
```

```
documents = [
    "NLP processes language",
    "NLP processes text",
    "Text mining analyses text"
]

vectorizer = TfidfVectorizer()

tfidf_matrix = vectorizer.fit_transform(documents)

tfidf_df = pd.DataFrame(
    tfidf_matrix.toarray(),
    columns=vectorizer.get_feature_names_out()
)

print(tfidf_df.round(3))
```

Display Important Terms

```
feature_names = vectorizer.get_feature_names_out()

for document_index, vector in enumerate(tfidf_matrix.toarray()):
    term_scores = list(zip(feature_names, vector))
    term_scores.sort(
        key=lambda item: item[1],
        reverse=True
    )

    print(f"\nDocument {document_index + 1}")

    for term, score in term_scores[:5]:
        print(term, round(score, 3))
```

Result

Documents were represented using TF-IDF scores.

Questions

83. Why are common words assigned lower weights?
84. What does document frequency mean?
85. How is TF-IDF different from Bag-of-Words?
86. Can TF-IDF capture word meaning?
87. Why is TF-IDF useful for document classification?

EXPERIMENT 13

Text Similarity Using Cosine Similarity

Aim

To calculate the similarity between text documents.

Theory

Cosine similarity measures the angle between two vectors: Cosine Similarity(A,B) = (A·B) / (||A|| ||B||).

Values usually range from "1" (highly similar) to "0" (unrelated); values close to "1" indicate greater similarity.

Program

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import pandas as pd

documents = [
    "Natural language processing analyses text",
    "NLP is used to process textual data",
    "Computer networks connect different devices"
]

vectorizer = TfidfVectorizer(
    stop_words="english"
)

tfidf_matrix = vectorizer.fit_transform(documents)

similarity_matrix = cosine_similarity(tfidf_matrix)

similarity_df = pd.DataFrame(
    similarity_matrix,
    index=["D1", "D2", "D3"],
    columns=["D1", "D2", "D3"]
)

print(similarity_df.round(3))
```

Result

Pairwise similarity between documents was calculated using TF-IDF and cosine similarity.

Questions

88. Why is cosine similarity suitable for text?
89. What does a similarity score of 1 indicate?
90. Can two semantically similar sentences have a low score?
91. How does preprocessing affect similarity?
92. Where is text similarity used?

EXPERIMENT 14

Text Classification Using Naive Bayes

Aim

To build a classical text-classification model.

Theory

Naive Bayes is a probabilistic classifier based on Bayes' theorem: P(C|X) = P(X|C)P(C) / P(X).

For text classification, Multinomial Naive Bayes is commonly used with word counts, term frequencies, or TF-IDF values.

Dataset Example

Text	Label
The product is excellent	Positive
I liked the service	Positive
The product is terrible	Negative
The service was poor	Negative

Program

```
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)
from sklearn.pipeline import Pipeline

texts = [
    "The product is excellent",
    "I liked the service",
    "The experience was wonderful",
    "The staff was helpful",
    "The product is terrible",
    "I disliked the service",
    "The experience was disappointing",
    "The staff was rude",
    "The quality is very good",
    "The quality is extremely poor"
]

labels = [
    "positive", "positive", "positive", "positive",
    "negative", "negative", "negative", "negative",
    "positive", "negative"
]

X_train, X_test, y_train, y_test = train_test_split(
    texts,
    labels,
    test_size=0.3,
```

```
    random_state=42,
    stratify=labels
)

model = Pipeline([
    (
        "tfidf",
        TfidfVectorizer(
            lowercase=True,
            ngram_range=(1, 2)
        )
    ),
    (
        "classifier",
        MultinomialNB()
    )
])

model.fit(X_train, y_train)

predictions = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, predictions))
print("\nClassification Report:")
print(classification_report(y_test, predictions))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, predictions))
```

Predict New Text

```
new_reviews = [
    "The service was excellent",
    "The product was disappointing"
]

predicted_labels = model.predict(new_reviews)

for text, label in zip(new_reviews, predicted_labels):
    print(text, "->", label)
```

Result

A traditional text classifier was developed using TF-IDF and Multinomial Naive Bayes.

Questions

93. Why is Naive Bayes suitable for text?
94. What is the independence assumption?
95. What is the role of TF-IDF?
96. Why should training and testing data be separated?
97. What problems occur with a very small dataset?

EXPERIMENT 15

Comparison of Classical Text Classifiers

Aim

To compare multiple machine-learning algorithms for text classification.

Models

- Multinomial Naive Bayes
- Logistic Regression
- Linear Support Vector Machine

Program

```
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score

texts = [
    "excellent product and good quality",
    "very satisfied with the service",
    "wonderful experience",
    "helpful staff and quick response",
    "poor quality product",
    "very bad service",
    "disappointing experience",
    "rude staff and delayed response",
    "good and reliable",
    "bad and unreliable",
    "happy with the product",
    "not satisfied with the product"
]

labels = [
    "positive", "positive", "positive", "positive",
    "negative", "negative", "negative", "negative",
    "positive", "negative", "positive", "negative"
]

X_train, X_test, y_train, y_test = train_test_split(
    texts,
    labels,
    test_size=0.33,
    random_state=42,
    stratify=labels
)

vectorizer = TfidfVectorizer(
    ngram_range=(1, 2),
    stop_words="english"
)
```

```
X_train_vector = vectorizer.fit_transform(X_train)
X_test_vector = vectorizer.transform(X_test)

models = {
    "Naive Bayes": MultinomialNB(),
    "Logistic Regression": LogisticRegression(
        max_iter=1000
    ),
    "Linear SVM": LinearSVC()
}

for model_name, model in models.items():
    model.fit(X_train_vector, y_train)

    predictions = model.predict(X_test_vector)

    accuracy = accuracy_score(
        y_test,
        predictions
    )

    print(model_name, ":", round(accuracy, 3))
```

Result

Different classical classifiers were trained and compared using the same text representation.

Questions

98. Why should all classifiers use the same train-test split?
99. Why is Linear SVM widely used for sparse text?
100. What is a hyperparameter?
101. Is accuracy sufficient for imbalanced data?
102. How can model selection be improved?

EXPERIMENT 16

End-to-End Text-Processing Pipeline

Aim

To design a reusable classical NLP processing pipeline.

Pipeline

Raw Input → Validation → Cleaning → Tokenisation → Stop-Word Removal → Lemmatisation → Feature Extraction → Model Training → Prediction → Evaluation

Program

```
import re
import pandas as pd
import nltk

from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
from sklearn.pipeline import Pipeline
from sklearn.base import BaseEstimator, TransformerMixin


class TextPreprocessor(
    BaseEstimator,
    TransformerMixin
):
    def __init__(self):
        self.stop_words = set(
            stopwords.words("english")
        )

        self.lemmatizer = WordNetLemmatizer()

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        processed_documents = []

        for text in X:
            processed_documents.append(
                self.preprocess(text)
            )

        return processed_documents

    def preprocess(self, text):
        if not isinstance(text, str):
            return ""

        text = text.lower()

        text = re.sub(
            r"http\S+|www\.\S+",
            " ",
            text
        )

        text = re.sub(
            r"[^a-z\s]",
            " ",
            text
        )
```

```
        text = re.sub(
            r"\s+",
            " ",
            text
        ).strip()

        tokens = word_tokenize(text)

        filtered_tokens = []

        for token in tokens:
            if (
                token not in self.stop_words
                and len(token) > 1
            ):
                lemma = self.lemmatizer.lemmatize(token)
                filtered_tokens.append(lemma)

        return " ".join(filtered_tokens)


data = {
    "text": [
        "The application is easy to use",
        "The software works very well",
        "The interface is excellent",
        "The application crashes frequently",
        "The software is extremely slow",
        "The interface is confusing",
        "The tool is useful and reliable",
        "The tool gives incorrect results"
    ],
    "label": [
        "positive", "positive", "positive",
        "negative", "negative", "negative",
        "positive", "negative"
    ]
}

df = pd.DataFrame(data)

X_train, X_test, y_train, y_test = train_test_split(
    df["text"],
    df["label"],
    test_size=0.25,
    random_state=42,
    stratify=df["label"]
)

pipeline = Pipeline([
    (
        "preprocessing",
        TextPreprocessor()
    ),
    (
        "tfidf",
        TfidfVectorizer(
```

```
            ngram_range=(1, 2)
        )
    ),
    (
        "classifier",
        LogisticRegression(
            max_iter=1000
        )
    )
])

pipeline.fit(X_train, y_train)

predictions = pipeline.predict(X_test)

print(classification_report(
    y_test,
    predictions
))
```

Advantages of a Pipeline

- Ensures identical processing during training and testing
- Reduces data leakage
- Simplifies experimentation
- Improves reproducibility
- Makes deployment easier

Result

An end-to-end classical NLP pipeline was implemented using preprocessing, TF-IDF and Logistic Regression.

---

11. Suggested Mini Projects

1. Medical Report Categorisation

Input: Short medical reports.

- Clean reports
- Preserve medical terms
- Perform tokenisation
- Extract common symptoms
- Classify reports by category

2. News Article Classification

Categories: sports, politics, technology, business, entertainment.

- TF-IDF
- Naive Bayes
- Logistic Regression
- Linear SVM

3. Student Feedback Analysis

Tasks:

- Read feedback from CSV
- Remove duplicate entries
- Identify frequently used words
- Analyse positive and negative feedback
- Classify feedback sentiment

4. Spam Email Detection

Features:

- Suspicious words
- URL frequency
- Punctuation count
- TF-IDF features
- Message length

5. Resume Skill Extraction

Tasks:

- Clean resume text
- Identify names and organisations
- Create a skill dictionary
- Extract technical skills
- Compare skills with a job description

6. College Notice Information Extraction

Extract: event name, date, time, venue, organiser, registration deadline. Use regular expressions, named-entity recognition, and rule-based patterns.

7. FAQ Retrieval Using Text Similarity

Steps:

103. Store predefined questions and answers
104. Clean the user question
105. Convert questions to TF-IDF vectors
106. Calculate cosine similarity
107. Return the answer linked to the most similar question

This is retrieval-based NLP and does not require an LLM.

12. General Dataset Structure

A basic supervised NLP dataset should contain:

document_id	text	label
1	The product is excellent	positive
2	The product is damaged	negative

Additional useful columns:

- Source
- Date
- Language
- Author
- Document length
- Category
- Preprocessing status

13. Dataset Inspection Program

```
import pandas as pd

df = pd.read_csv("dataset.csv")

print("Shape:", df.shape)

print("\nColumns:")
print(df.columns)

print("\nMissing values:")
print(df.isnull().sum())
```

```
print("\nDuplicate rows:")
print(df.duplicated().sum())

print("\nClass distribution:")
print(df["label"].value_counts())

df["word_count"] = (
    df["text"]
    .fillna("")
    .str.split()
    .str.len()
)

df["character_count"] = (
    df["text"]
    .fillna("")
    .str.len()
)

print(df[
    [
        "text",
        "word_count",
        "character_count"
    ]
].head())
```

14. Common NLP Errors

14.1 Data Leakage

Incorrect: fitting the vectoriser on all documents before splitting the data:

```
vectorizer.fit_transform(all_documents)  # before splitting the data
```

Correct sequence:

108. Split the data
109. Fit the vectoriser only on training data
110. Transform training data
111. Transform testing data

14.2 Over-Cleaning

Removing all numbers from financial or medical text may destroy useful information, e.g. "Blood pressure: 140/90" or "Revenue increased by 20%".

14.3 Removing Negation

The sentences "The product is good." and "The product is not good." should not produce identical representations.

14.4 Ignoring Class Imbalance

Suppose 95% of messages are non-spam and 5% are spam. A model predicting every message as non-spam achieves 95% accuracy but is useless.

14.5 Using Only Training Accuracy

Training accuracy does not indicate generalisation. Always report testing or validation performance.

14.6 Applying the Wrong Preprocessing

The preprocessing method must depend on the application:

Application	Important Elements
Sentiment analysis	Negation, punctuation, emojis
Financial NLP	Numbers, currency, percentages
Medical NLP	Abbreviations, measurements, drug names
Social media	Hashtags, mentions, emojis
Search system	Lemmas, synonyms, keywords

15. Model Evaluation

Confusion Matrix

For binary classification:

	Predicted Positive	Predicted Negative
Actual Positive	True Positive	False Negative
Actual Negative	False Positive	True Negative

Accuracy

Accuracy = (TP + TN) / (TP + TN + FP + FN)

Precision

Precision = TP / (TP + FP)

Recall

Recall = TP / (TP + FN)

F1-Score

F1 = 2 × (Precision × Recall) / (Precision + Recall)

16. Error Analysis

Students should not stop after reporting accuracy. For every experiment, analyse:

112. Which documents were classified incorrectly?
113. Were the documents too short?
114. Did they contain unseen words?
115. Did preprocessing remove important information?
116. Was the label ambiguous?
117. Were negation and sarcasm present?
118. Was the dataset balanced?
119. Were there spelling errors?
120. Did the model depend on irrelevant words?
121. Could bigrams improve the result?

Error Analysis Program

```
results = pd.DataFrame({
    "text": X_test,
    "actual": y_test,
    "predicted": predictions
})
```

```
errors = results[
    results["actual"] != results["predicted"]
]

print(errors)
```

17. Suggested Laboratory Sequence

Week	Experiment
1	Python strings and text data structures
2	Reading TXT, CSV and JSON data
3	Regular expressions and text cleaning
4	Sentence and word tokenisation
5	Stop words and frequency analysis
6	Stemming and lemmatisation
7	POS tagging and chunking
8	Named-entity recognition
9	Dependency parsing
10	Bag-of-Words and n-grams
11	TF-IDF and document similarity
12	Naive Bayes text classification
13	Logistic Regression and SVM
14	Evaluation and error analysis
15	End-to-end mini-project demonstration

18. Laboratory Assessment Scheme

Component	Marks
Understanding of problem	10
Algorithm and workflow	10
Python implementation	25
Correctness of output	15
Interpretation and observations	15
Viva voce	10
Journal documentation	10

Extension activity	5
Total	100

19. Journal-Writing Format

For each practical, students should include:

Experiment Number and Title

Clearly mention the experiment.

Aim

Write the objective in one or two sentences.

Theory

Explain the main concept and its importance.

Input

Mention input source, file format, number of documents, columns, and sample text.

Algorithm

Write the processing stages sequentially.

Program

Include well-formatted and commented Python code.

Output

Include sample output, table, chart (where useful), and evaluation metrics.

Observation

Interpret what happened during processing.

Result

State whether the objective was achieved.

Limitations

Mention at least one limitation.

Extension

Suggest one improvement or alternative method.

20. General Viva Questions

122. What is Natural Language Processing?
123. What is the difference between structured and unstructured data?
124. What is a corpus?
125. What is tokenisation?
126. Why is text normalisation required?
127. What is the difference between stemming and lemmatisation?
128. What is a stop word?
129. What is a vocabulary?

130. What is a document-term matrix?
131. Why are text matrices sparse?
132. What is Bag-of-Words?
133. What is an n-gram?
134. What is TF-IDF?
135. What is cosine similarity?
136. What is POS tagging?
137. What is named-entity recognition?
138. What is dependency parsing?
139. What is a feature vector?
140. Why should data be divided into training and testing sets?
141. What is data leakage?
142. What is class imbalance?
143. What is precision?
144. What is recall?
145. What is an F1-score?
146. Why is error analysis important?
147. What is a pipeline?
148. What is the limitation of classical NLP?
149. How does domain language affect NLP?
150. Why can sarcasm be difficult to process?
151. Why should preprocessing depend on the application?

21. Final Integrated NLP Workflow

```
def classical_nlp_workflow(
    input_file,
    text_column,
    label_column
):
    """
    General outline of a classical NLP workflow.
    """

    import pandas as pd

    # Step 1: Read data
    df = pd.read_csv(input_file)

    # Step 2: Validate required columns
    required_columns = {
        text_column,
        label_column
    }

    if not required_columns.issubset(df.columns):
        raise ValueError(
            "Required columns are missing."
        )

    # Step 3: Remove missing records
    df = df.dropna(
        subset=[
            text_column,
            label_column
        ]
    )

    # Step 4: Remove duplicates
    df = df.drop_duplicates(
        subset=[text_column]
    )

    # Step 5: Inspect data
    print("Number of records:", len(df))
    print("\nClass distribution:")
    print(df[label_column].value_counts())

    return df
```

22. Core Principle

Classical NLP is not simply a collection of library functions. A meaningful NLP experiment requires students to understand:

- What the input contains
- Why a processing operation is needed
- What information may be lost

- How text is represented numerically
- Why a model produces a particular output
- How performance should be evaluated
- Where the method may fail

The central learning cycle is:

Observe the language → Represent it carefully → Process it logically → Evaluate the output → Analyse the errors

23. Recommended Final Practical

Title

Design and Implement a Classical NLP System for a Selected Application

Possible Applications

- Medical-report categorisation
- Student-feedback analysis
- Spam detection
- News classification
- Complaint classification
- FAQ retrieval
- Resume skill extraction
- Notice information extraction
- Document similarity
- Product-review sentiment analysis

Required Components

152. Problem definition
153. Dataset description
154. Text-input method
155. Dataset inspection
156. Cleaning
157. Tokenisation
158. Normalisation
159. Linguistic analysis
160. Feature extraction
161. Model selection
162. Evaluation
163. Error analysis
164. Limitations
165. Conclusion

Expected Deliverables

- Python notebook
- Processed dataset
- Laboratory journal
- Result table
- Confusion matrix
- Sample predictions
- Error-analysis table
- Short project presentation

Conclusion

This laboratory manual develops NLP progressively from Python text handling to an end-to-end classical machine-learning system.

It enables students to understand how language data moves through the complete pipeline:

Input → Data Structure → Cleaning → Tokens → Linguistic Features → Numerical Vectors → Model → Evaluation

The focus remains on transparent, interpretable and foundational NLP methods that formed the standard NLP workflow before the widespread use of large language models.